

S.Y.B.SC.(DATA SCIENCE) SEM- III

Subject: Data Structure – I

Unit1 : Introduction to Data Structures and Algorithm Analysis

By,

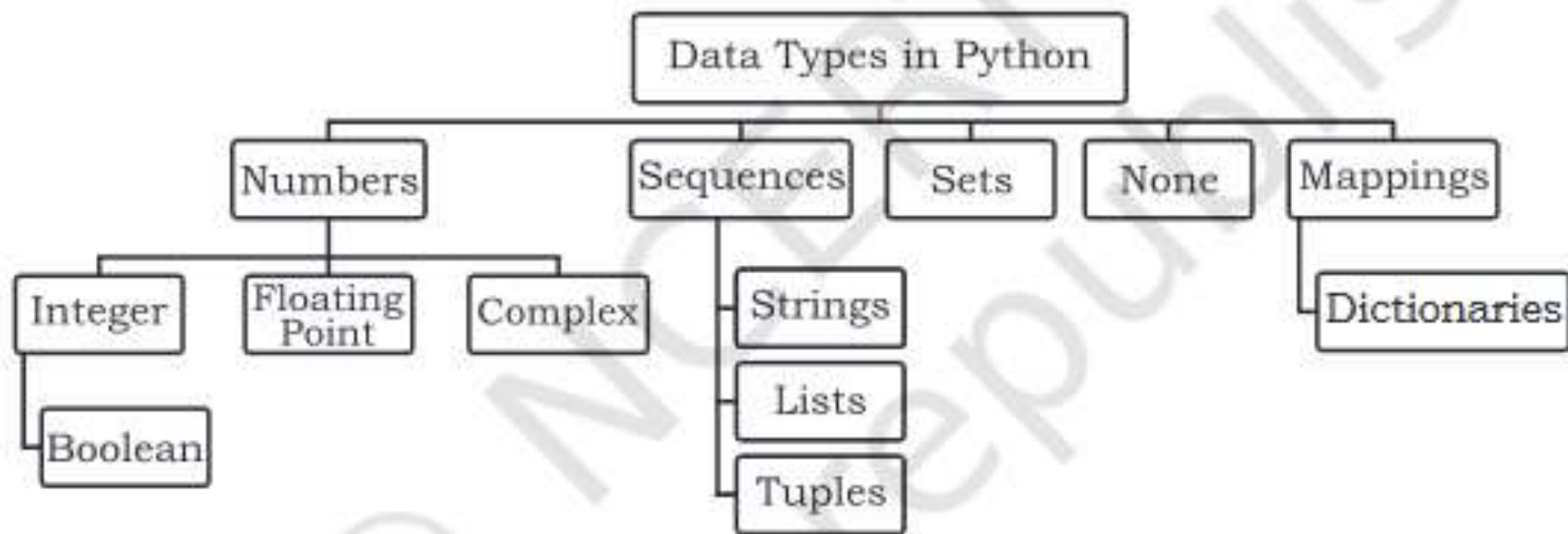
Mrs. Reshma Masurekar

Dr. D. Y. Patil ACS College,

Pimpri, Pune 18.

Python Data Types

- Every value has a datatype, and variables can hold values.
- The following is a list of the Python-defined data types.
 - Numeric Types: int, float, complex
 - Text Type: str
 - Boolean Type :bool
 - Sequence Types: list, tuple, range
 - Mapping Type: dict
 - Set Types :set
 - Binary Types: bytes, bytearray
 - None Type: None



Numeric Types: int, float, complex

- **int, or integer**, is a whole number, positive or negative, without decimals, of unlimited length.
- **Float**, or "floating point number" is a number, positive or negative, containing one or more decimals.
- **Complex numbers** A complex number contains the real and imaginary parts. i.e., $a + ib$.
- **Example:**

int:

x = 10

y =

35656333445889911

z = -3166333

Float

x = 4.50

y = 5.0

z = -15.28

Complex

x = 5+4j

y = 7j

z = -5j

Text Type: str

- In Python, a **string is a sequence of characters**.
- Strings in python are surrounded by either single quotation marks, or double quotation marks.
- 'Science' is the same as "Science".

- **Example:**

```
str = "Science"  
print(str)
```

```
str = "Data"+"Science"  
print(str)
```

- operator + is used to combine two strings.
- **Example:** "Data"+" Science" returns "Data Science"

Boolean: bool

- True and False are the two default values for the Boolean type.
- False can be represented by the 0 or the letter "F," while true can be represented by any value that is not zero.

- **Example:**

```
# Python program to check the boolean type
```

```
print(type(True))
```

```
print(type(False))
```

- **Output:**

```
<class 'bool'>
```

```
<class 'bool'>
```

List

- Lists in Python are like arrays in C, but lists can contain data of different types.
- A list is created by placing elements inside **square brackets []**, separated by commas.
- **Ordered** - They maintain the order of elements.
- **Mutable** - Items can be changed after creation.
- **Allow duplicates** - They can contain duplicate values.
- **Example:**
a list of three elements
ages = [19, 26, 29]
print(ages)
Output: [19, 26, 29]

Tuple

- A Tuple is like a list.
- Tuples, also contain a collection of items from various data types.
- A tuple is created by placing elements inside a round **brackets ()** separated by commas.
- **Ordered** - They maintain the order of elements.
- **Immutable** - They cannot be changed after creation.
- **Allow duplicates** - They can contain duplicate values.
- **Example:**
 numbers = (1, 2, -5)
 print(numbers)
 # Output: (1, 2, -5)

Mapping Type: Dictionary

- A Python dictionary is a collection of items, similar to lists and tuples.
- A dictionary is a **key-value pair** set **arranged in any order**.
- It stores a specific value for each key.
- a dictionary is created by placing key: value pairs inside curly **brackets {}**, separated by commas.

- **Example:**

```
d = {1:'Arun', 2:'Varun', 3:'Ram', 4:'Raj'}  
print (d)
```

- **Output:**

```
{1:'Arun', 2:'Varun', 3:'Ram', 4:'Raj'}
```

Set

- A Set is a **unordered** collection of **unique** items enclosed within the curly braces{, }.
- A set is a collection of **unique** data, meaning that elements within a set **cannot be duplicated**.
- **Unordered** collection: The elements of a set have no set order; Set items can appear in a different order every time while using it.
- **Mutable** (can change after creation)
- In Python, set is created by placing all the elements inside curly braces {}, separated by commas.
- **Example:**

```
set1={'C','C++','Java','Python','Java'}  
print(set1)
```

Binary types

- In Python, bytes and bytearray are used to work with binary data and memory views of binary data.
- The **bytes** is an **immutable** sequence type to represent sequences of bytes (8-bit values). Each element in a bytes object is an integer in the range [0, 255].
- Bytes are defined using the b prefix followed by a sequence of bytes enclosed in single or double quotes.
- Example: `byte1 = b'Data Science'`
- The **bytearray** is similar to **bytes** but bytearray objects can be modified after creation.
- Example: `bytearray1= bytearray([72, 101, 108, 108, 111])`

- A memoryview type to create a “view” of memory containing binary data. It doesn’t store the data itself but provides a view into the memory where the data is stored.

- Example

```
data = bytearray([1, 2, 3, 4, 5])
```

```
mem_view = memoryview(data)
```

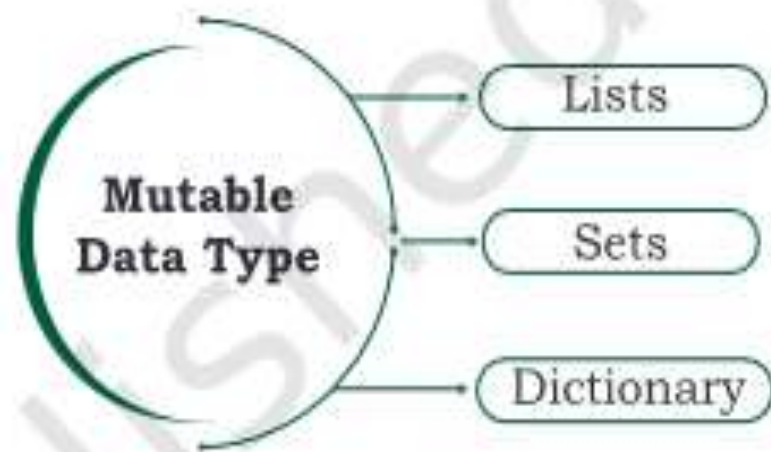
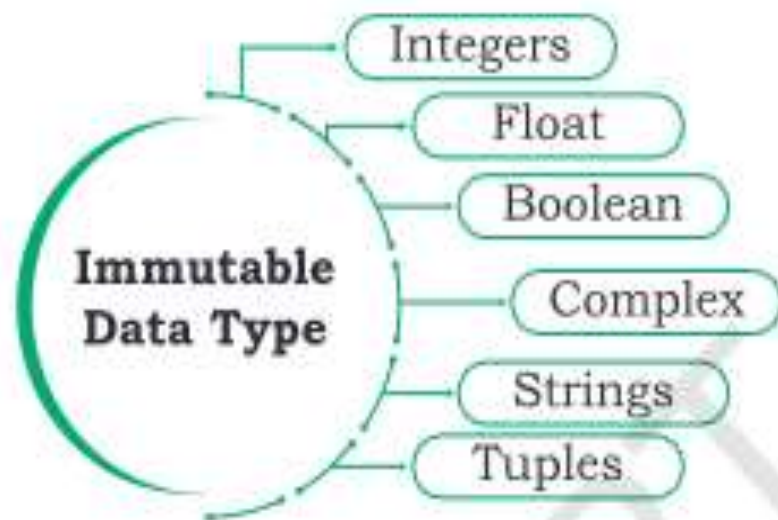
None

- The ***None*** represents a special value indicating the absence of a value.
- Example: `no_value = None`

- Mutable data types in Python
 - List
 - Set
 - Dictionary
- Immutable data types in Python
 - Number
 - String
 - Tuple

Mutable and Immutable Data Types

- Variables whose values can be changed after they are created and assigned are called **mutable**.
- Variables whose values cannot be changed after they are created and assigned are called **immutable**.
- When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.



String	List	Tuple	Set	Dictionary
Immutable	Mutable	Immutable	Mutable	Mutable
Ordered/Indexed	Ordered/Indexed	Ordered/Indexed	Unordered	Unordered
Allows Duplicate Members	Allows Duplicate Members	Allows Duplicate Members	Doesn't allow Duplicate Members	Doesn't allow Duplicate Members
Empty String=""	Empty list=[]	Empty tuple=()	Empty Set=set()	Empty dictionary={ }
String with single element="H"	List with single item=["Hello"]	Tuple with single item=("Hello")	Set with single item={"Hello"}	Dictionary with single item={"Hello":1}
It can store data type string only.	It can store any data type str, list, set, tuple, int, dic	It can store any data type str, list, set, tuple,	It can store data types (int, str, tuple) but not	Inside of dictionary key can be int, str and tuple o can be

range()

- The [range](#) type represents an immutable sequence of numbers and is commonly used for looping a specific number of times in [for](#) loops.

- **Syntax:**

`range(stop)`

`range(start, stop)`

`range(start, stop[, step])`

Note: If the *step* argument is omitted, it defaults to 1. and If the *start* argument is omitted, it defaults to 0

- **Example:**

`list(range(10))` `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

`list(range(1, 11))` `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`

`list(range(0))` `[]`